



声明：百科词条人人可编辑，词条创建和修改均免费，绝不存在官方及代理商付费代编，请勿上当受骗。 [详情>>](#)



首页

分类

特色百科

用户

权威合作

手机百科

个人中心

zlib

编辑

zlib是提供**数据压缩**用的函式库，由Jean-loup Gailly与Mark Adler所开发，初版0.9版在1995年5月1日发表。zlib使用DEFLATE算法，最初是为libpng函式库所写的，后来普遍为许多软件所使用。此函式库为自由软件，使用zlib授权。截至2007年3月，zlib是包含在Coverity的**美国国土安全部**赞助者选择继续审查的开源项目。

中文名	zlib	定 义	提供 数据压缩 用的函数库
	外文名	开 发	Jean-loup Gailly与Mark Adler

zlib

目录	1 特性 - 数据头(header) - 算法 - 使用资源 2 业界应用	3 使用范例 4 zlib许可
----	---	--

特性



编辑

数据头(header)

zlib能使用一个gzip数据头，zlib数据头或者不使用数据头压缩数据。

通常情况下，**数据压缩**使用zlib数据头，因为这提供错误数据检测。当数据不使用数据头写入时，结果是没有任何错误检测的原始DEFLATE数据，那么**解压缩软件**的调用者不知道压缩数据在什么地方结束。

gzip数据头比zlib数据头要大，因为它保存了文件名和其他文件系统信息，事实上这是广泛使用的gzip文件的数据头格式。注意zlib函式库本身不能创建一个gzip文件，但是它相当轻松的通过把压缩数据写入到一个有**gzip文件头**的文件中。

算法

目前zlib仅支持一个LZ77的变种算法，DEFLATE的算法。

这个算法使用很少的系统资源，对各种数据提供很好的压缩效果。这也是在ZIP档案中无一例外的使用这个算法。（尽管zip文件格式也支持几种其他的算法）。

看起来zlib格式将不会被扩展使用任何其他算法，尽管数据头可以有这种可能性。

使用资源

函数库提供了对处理器和内存使用控制的能力

不同的压缩级别数值可以指示不同的压缩执行速度。

还有内存控制管理的功能。这在一些诸如**嵌入式系统**这样内存有限制的环境中是有用的。

策略



zlib图册

V百科

往期回顾



分享



相关软件

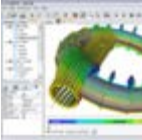
纠错



clam anti...



openbsd



wxwidgets



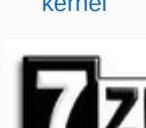
kernel



navicat



winscp



7-zip



setuptools



openwrt

相关工具

纠错



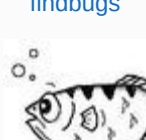
findbugs



dnsmasq



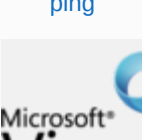
ping



gdb



winrar



microsoft v...

计算机术语

纠错



lzw算法



mono



root

压缩可以针对特定类型的数据进行优化

如果你总是使用**zlib**库压缩压缩特定类型的数据，那么可以使用有针对性的策略可以提高压缩效率和性能。例如，如果你的数据包含很长的重复数据，那么可以用**RLE**（运行长度编码）策略，可能会有更好的结果。

对于一般的数据，默认的策略是首选。

错误处理

错误可以被发现和跳过

数据混乱可以被检测（只要数据和**zlib**或者**gzip**数据头一起被写入－参见上面）

此外，如果全刷新点（**full-flush points**）被写入到压缩后的数据流中，那么错误数据是可以被跳过的，并且**解压缩**将重新同步到下个全刷新点。（错误数据的无错恢复被提供）。全刷新点技术对于在不可靠的通道上的大数据流是很有用的，一些过去的的数据丢失是不重要的（例如多媒体数据），但是建立太多的全刷新点会极大的影响速度和压缩。

数据长度

对于压缩和**解压缩**，没有数据长度的限制

重复调用**库函数**允许处理无限的数据**数据块**。一些辅助代码（计数变量）可能会溢出，但是不影响实际的压缩和解压缩。

当压缩一个长（无限）数据流时，最好写入全刷新点。

业界应用

今天，**zlib**是一种事实上的业界标准，以至于在标准文档中，**zlib**和**DEFLATE** 常常互换使用。数以千计的应用程序直接或间接依靠**zlib**压缩函式库，包括：

- * **Linux**核心：使用**zlib**以实作网络协定的压缩、档案系统的压缩以及开机时**解压缩**自身的核心。
- * **libpng**，用于**PNG**图形格式的一个实现，对**bitmap**数据规定了**DEFLATE**作为流压缩方法。
- * **Apache**：使用**zlib**实作**http 1.1**。
- * **OpenSSH**、**OpenSSL**：以**zlib**达到最佳化加密网络传输。
- * **FFmpeg**：以**zlib**读写**Matroska**等以**DEFLATE**算法压缩的多媒体串流格式。
- * **rsync**：以**zlib**最佳化远端同步时的传输。
- * The **dpkg** and **RPM** package managers, which use **zlib** to unpack files from compressed software packages.
- * **Subversion** 、**Git**和 **CVS** **版本控制** 系统，使用**zlib**来压缩和远端仓库的通讯流量。
- * **dpkg**和**RPM**等包管理软件：以**zlib****解压缩****RPM**或者其他**封包**。

因为其代码的可移植性，宽松的许可以及较小的内存占用，**zlib**在许多**嵌入式设备**中也有应用。

使用范例

以下代码可直接用于解压HTTP gzip

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <errno.h>
```

```
#include <zlib.h>
```

```
/* Compress data */
```



词条统计

浏览次数：128756次

编辑次数：20次[历史版本](#)

最近更新：2017-08-02

创建者：[tyeken8](#)

- | | |
|-------------------|--------------------|
| 1 房产中介加盟 | 12 400电话是什么 |
| 2 井水净水器 | 13 世界豪华车排名 |
| 3 ui设计都学什么 | 14 马尔代夫岛屿排 |
| 4 在职研究生取消 | 15 泡沫板生产设备 |
| 5 排水管价格表 | 16 域名注册 |
| 6 挖机培训 | 17 英语绕口令练习 |
| 7 公路洒水车 | 18 软膜天花 |
| 8 不用审核的贷款 | 19 我怎么做推广 |
| 9 一级消防工程师 | 20 自考和成考的区 |
| 10 哪里有私人贷款 | 21 全国十大净水器 |
| 11 不爱硬 | 22 人力资源资格证 |

[编辑](#)

[编辑](#)

```
int zcompress(Bytef *data, uLong ndata,

Bytef *zdata, uLong *nzdata)

{

z_stream c_stream;

int err = 0;

if(data && ndata > 0)

{

c_stream.zalloc = (alloc_func)0;

c_stream.zfree = (free_func)0;

c_stream.opaque = (voidpf)0;

if(deflateInit(&c_stream, Z_DEFAULT_COMPRESSION) != Z_OK) return -1;

c_stream.next_in = data;

c_stream.avail_in = ndata;

c_stream.next_out = zdata;

c_stream.avail_out = *nzdata;

while (c_stream.avail_in != 0 && c_stream.total_out < *nzdata)

{

if(deflate(&c_stream, Z_NO_FLUSH) != Z_OK) return -1;

}

if(c_stream.avail_in != 0) return c_stream.avail_in;

for (;;) {

if((err = deflate(&c_stream, Z_FINISH)) == Z_STREAM_END) break;

if(err != Z_OK) return -1;

}

if(deflateEnd(&c_stream) != Z_OK) return -1;

*nzdata = c_stream.total_out;

return 0;

}

return -1;

}

/* Compress gzip data */

int gzcompress(Bytef *data, uLong ndata,

Bytef *zdata, uLong *nzdata)

{

z_stream c_stream;

int err = 0;

if(data && ndata > 0)

{

c_stream.zalloc = (alloc_func)0;
```

```
c_stream.zfree = (free_func)0;

c_stream.opaque = (voidpf)0;

if(deflateInit2(&c_stream, Z_DEFAULT_COMPRESSION, Z_DEFLATED,
-MAX_WBITS, 8, Z_DEFAULT_STRATEGY) != Z_OK) return -1;

c_stream.next_in = data;

c_stream.avail_in = ndata;

c_stream.next_out = zdata;

c_stream.avail_out = *nzdata;

while (c_stream.avail_in != 0 && c_stream.total_out < *nzdata)

{

if(deflate(&c_stream, Z_NO_FLUSH) != Z_OK) return -1;

}

if(c_stream.avail_in != 0) return c_stream.avail_in;

for (;;) {

if((err = deflate(&c_stream, Z_FINISH)) == Z_STREAM_END) break;

if(err != Z_OK) return -1;

}

if(deflateEnd(&c_stream) != Z_OK) return -1;

*nzdata = c_stream.total_out;

return 0;

}

return -1;

}

/* Uncompress data */

int zdecompress(Byte *zdata, uLong nzdata,

Byte *data, uLong *ndata)

{

int err = 0;

z_stream d_stream; /* decompression stream */

d_stream.zalloc = (alloc_func)0;

d_stream.zfree = (free_func)0;

d_stream.opaque = (voidpf)0;

d_stream.next_in = zdata;

d_stream.avail_in = 0;

d_stream.next_out = data;

if(inflateInit(&d_stream) != Z_OK) return -1;

while (d_stream.total_out < *ndata && d_stream.total_in < nzdata) {

d_stream.avail_in = d_stream.avail_out = 1; /* force small buffers */
```

```
if((err = inflate(&d_stream, Z_NO_FLUSH)) == Z_STREAM_END) break;

if(err != Z_OK) return -1;

}

if(inflateEnd(&d_stream) != Z_OK) return -1;

*ndata = d_stream.total_out;

return 0;

}

/* HTTP gzip decompress */

int httpgzdecompress(Byte *zdata, uLong nzdata,

Byte *data, uLong *ndata)

{

int err = 0;

z_stream d_stream = {0}; /* decompression stream */

static char dummy_head[2] =

{

0x8 + 0x7 * 0x10,

(((0x8 + 0x7 * 0x10) * 0x100 + 30) / 31 * 31) & 0xFF,

};

d_stream.zalloc = (alloc_func)0;

d_stream.zfree = (free_func)0;

d_stream.opaque = (voidpf)0;

d_stream.next_in = zdata;

d_stream.avail_in = 0;

d_stream.next_out = data;

if(inflateInit2(&d_stream, 47) != Z_OK) return -1;

while (d_stream.total_out < *ndata && d_stream.total_in < nzdata) {

d_stream.avail_in = d_stream.avail_out = 1; /* force small buffers */

if((err = inflate(&d_stream, Z_NO_FLUSH)) == Z_STREAM_END) break;

if(err != Z_OK )

{

if(err == Z_DATA_ERROR)

{

d_stream.next_in = (Bytef*) dummy_head;

d_stream.avail_in = sizeof(dummy_head);

if((err = inflate(&d_stream, Z_NO_FLUSH)) != Z_OK)

{

return -1;

}

}

}
```



```
else return -1;

}

}

if(inflateEnd(&d_stream) != Z_OK) return -1;

*ndata = d_stream.total_out;

return 0;

}

/* Uncompress gzip data */

int gzdecompress(Byte *zdata, uLong nzdata,

Byte *data, uLong *ndata)

{

int err = 0;

z_stream d_stream = {0}; /* decompression stream */

static char dummy_head[2] =

{

0x8 + 0x7 * 0x10,

(((0x8 + 0x7 * 0x10) * 0x100 + 30) / 31 * 31) & 0xFF,

};

d_stream.zalloc = (alloc_func)0;

d_stream.zfree = (free_func)0;

d_stream.opaque = (voidpf)0;

d_stream.next_in = zdata;

d_stream.avail_in = 0;

d_stream.next_out = data;

if(inflateInit2(&d_stream, -MAX_WBITS) != Z_OK) return -1;

//if(inflateInit2(&d_stream, 47) != Z_OK) return -1;

while (d_stream.total_out < *ndata && d_stream.total_in < nzdata) {

d_stream.avail_in = d_stream.avail_out = 1; /* force small buffers */

if((err = inflate(&d_stream, Z_NO_FLUSH)) == Z_STREAM_END) break;

if(err != Z_OK )

{

if(err == Z_DATA_ERROR)

{

d_stream.next_in = (Bytef*) dummy_head;

d_stream.avail_in = sizeof(dummy_head);

if((err = inflate(&d_stream, Z_NO_FLUSH)) != Z_OK)

{

return -1;
```

```

}

else return -1;

}

}

if(inflateEnd(&d_stream) != Z_OK) return -1;

*ndata = d_stream.total_out;

return 0;

}

#ifdef _DEBUG_ZSTREAM

#define BUF_SIZE 65535

int main()

{

char *data = "kjDalkfjdflkjdlkfjdlkfjldkfjldkfjlddajfkdfjkdfaskf;ldsfk;ldakf;ldskfl;dskf;ld";

uLong ndata = strlen(data);

Bytef zdata[BUF_SIZE];

uLong nzdata = BUF_SIZE;

Bytef odata[BUF_SIZE];

uLong nodata = BUF_SIZE;

memset(zdata, 0, BUF_SIZE);

//if(zcompress((Bytef *)data, ndata, zdata, &nzdata) == 0)

if(gzcompress((Bytef *)data, ndata, zdata, &nzdata) == 0)

{

fprintf(stdout, "nzdata:%d %s\n", nzdata, zdata);

memset(odata, 0, BUF_SIZE);

//if(zdecompress(zdata, ndata, odata, &nodata) == 0)

if(gzdecompress(zdata, ndata, odata, &nodata) == 0)

{

fprintf(stdout, "%d %s\n", nodata, odata);

}

}

}

#endif

```

zlib许可

□ 编辑

zlib许可^[1]是一个自由软件授权协议，但并非copyleft。Box2D就使用了zlib许可。

参考资料

- ## 1. zlib许可原文

☐ 猜你喜欢

- [战痕天下](#)
- [大学生信用贷款](#)
- [春秋旅游官方网](#)
- [suv汽车大全](#)
- [中超风云](#)
- [东莞二手车](#)
- [氟碳漆](#)
- [淘宝店铺装修教程](#)
- [宜兴紫砂壶](#)
- [征途怀旧版](#)



ui设计都学什么



怎么让男友硬



简欧风格



房产中介加盟



马尔代夫岛屿排名



自考和成考的区别



全屋定制



广告

☐ 新手上路

- [成长任务](#)
- [编辑入门](#)
- [编辑规则](#)
- [百科术语](#)

☐ 我有疑问

- [我要质疑](#)
- [在线客服](#)
- [参加讨论](#)
- [意见反馈](#)

☐ 投诉建议

- [举报不良信息](#)
- [未通过词条申诉](#)
- [投诉侵权信息](#)
- [封禁查询与解封](#)

©2017 Baidu [使用百度前必读](#) | [百科协议](#) | [百度百科合作平台](#) | 京ICP证030173号

 [京公网安备11000002000001号](#)